

Choir: An Open Protocol for Distributed Multi-Agent Autoformalization

Yidi Qi
Harvard University
yidi_qi@seas.harvard.edu

Melanie Weber
Harvard University
mweber@seas.harvard.edu



Abstract

AI agents can now formalize entire textbooks and major theorems in proof assistants such as Lean, but current efforts are typically centralized: a single team runs all agents and bears the full computational cost. We introduce *Choir*, an open protocol for distributed formalization. Choir decomposes a project into tasks that can be completed by independent contributors, each running their own agent with their own LLM subscription, while coordinating entirely through the project’s GitHub repository. To support open participation, every contribution is checked by a deterministic gate before merge. Choir supports Lean 4, Isabelle, and Rocq, and is open source and modular, allowing projects to replace individual components or extend the protocol.

1 Introduction

Proof assistants such as Lean 4 [1], Isabelle [2], and Rocq [3] check every step of a proof against a small trusted kernel, so a formalized result can be relied on without re-reading its proof. Communities maintain shared libraries on top of them, such as the Lean libraries Mathlib [4], Physlib [5, 6] and CSLib [7] for mathematics, physics and computer science, which new formalizations build on. Until recently this work was done almost entirely by hand, and building these libraries took many contributors years.

AI agents have changed the pace. In the past year they have formalized whole textbooks. A Meta FAIR team formalized a graduate algebraic combinatorics textbook into roughly 130,000 lines of Lean in one week, using about 30,000 agent runs working in parallel at an estimated cost of \$100K with prompt caching and \$430K without [8]. The same group has since built Atlas, a library formalized from 26 textbooks [9]. This effort is part of a broader shift towards large-scale, agent-driven formalization across mathematics [10–13]. Agents have also completed major research-level formalizations. Math Inc.’s Gauss finished the formal proofs of sphere packing in dimensions 8 and 24, the first building on a human-led project [14, 15]. Most recently, a team of Claude agents at Anthropic completed a formal proof of Fermat’s Last Theorem in Lean in under two weeks [16]. Most of these systems are built for a single team that runs every agent on its own budget, and several operate at a scale out of reach for a typical academic group.

The open mathematical community has what such efforts need: expertise, and a large amount of agent capacity, already paid for and spread across individual LLM subscriptions. If that capacity could be pooled, large-scale formalization would no longer depend on one well-funded team. Concurrent projects such as *TauCeti* [17] and *Prove2Me* [18] explore this idea (Section 5).

We present *Choir*, an open protocol for distributed formalization. Choir decomposes a project into tasks that can be completed by independent contributors, each running their own agent with their own LLM subscription, while coordinating entirely through the project’s GitHub repository. The project’s overseer runs an orchestrator agent that plans the project, writes the theorem statements, and publishes them as tasks in a GitHub repository. Contributors claim tasks, prove them on their own machines, and submit pull requests. Since contributors may be strangers running unknown models, a deterministic verification gate checks each pull request, including that the published statement is unchanged, before the orchestrator reviews and merges it. Choir currently supports Lean 4, Isabelle, and Rocq. It is completely open source and designed to be modular: a project can swap in its own planner or orchestrator, and contributors can use any agent. Our hope is that others will be able to build on Choir, or combine it with their own workflows.

Contributions. This paper makes three technical contributions:

- (1) **A task distribution and management system** (Section 2) that distributes a project’s formalization across many contributors’ agents through its own GitHub repository, and lets the proof tree grow from the decompositions they send back.
- (2) **A deterministic verification gate** (Section 3) that re-checks every pull request from source mechanically.
- (3) **A history visualizer** (Section 4) that replays how a project’s proof tree grew and which contributor proved or decomposed each node, reconstructed from its git log, issues, and pull requests alone.

We tested Choir by formalizing Yufei Zhao’s lecture notes *Probabilistic Methods in Combinatorics* [19] in Lean 4. The same notes are among the textbooks in Meta FAIR’s Atlas [9], which makes a direct comparison possible. The result is at <https://github.com/yidiq7/ProbMethodCombinatorics>, and the code of Choir is available under the Apache-2.0 license at <https://github.com/Weber-GeoML/Choir>.

2 Task distribution and management

Choir splits a project across four parties (Table 1, Figure 1): a human overseer, two kinds of LLM agents, and the repository itself. This section follows one unit of work from the orchestrator’s plan to a merged pull request, naming the commands each step runs.

Choir manages the file edits and the version control between multiple agents through GitHub, which already supplies the tools a coordination platform needs: identity, event triggers, access control, public audit, etc. The commands in Table 2 bound what an agent may do. They are implemented as Python wrappers over GitHub’s `gh` CLI. The orchestrator’s procedures are written in a playbook document, so any agent that can read the playbook and run these commands can serve as the orchestrator.

Role	Runs where	Does
Overseer	— (human)	Owens the project. Sets the goal, the audit policy, and how far the orchestrator may act alone, and rules on new axioms.
Orchestrator	overseer’s machine, LLM account, and GitHub auth	Plans and decomposes the goal, <i>authors the statements</i> , publishes typed tasks, reviews and merges pull requests, replans in a loop.
Workers	contributors’ machines and accounts	Claim a task, prove it with the contributor’s own agent, submit. Hold <i>no</i> repository write access: a claim is a comment, a submission is a pull request from the contributor’s own fork.
Gate	GitHub Actions in the project repo	Deterministic audits and intake validation.

Table 1: The four roles.

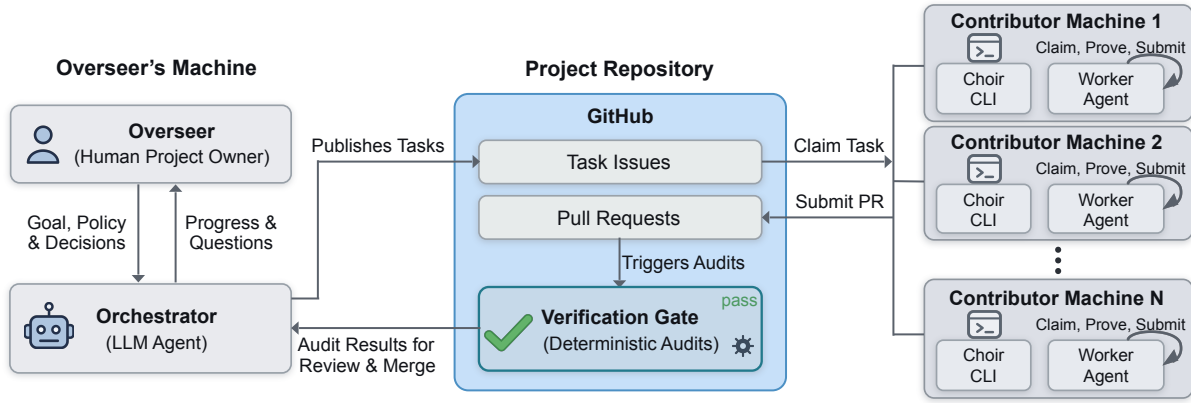


Figure 1: The orchestrator publishes tasks as GitHub issues, and workers on contributors’ own machines claim them and submit pull requests. The gate audits each pull request before the orchestrator reviews and merges it.

Plan. The orchestrator opens a project by writing a roadmap into the repository: the goal, the route it intends to take, its own reasoning log, and a dependency graph as JSON. A node is publishable once it and every dependency it names have been formally stated. Decomposition of theorems at this stage is deliberately shallow to reduce the burden on the orchestrator. The orchestrator states only the skeleton that carries the mathematical structure and leaves further decomposition to the workers.

The planner is modular: the overseer can plug in their own planner or human-written files, and use Choir only for task management, as long as the roadmap follows the same format.

Publish. `choir orch create-task` turns a ready node into a GitHub issue whose body opens with a YAML record. Each task is a statement whose proof is a placeholder, labeled by the orchestrator with a difficulty and a priority level. The overseer may cap the difficulty a given worker will take based on which model they use, both saving costs and keeping weaker models off work they are unlikely to finish.

Claim. A contributor working from another account holds no write access to the project repository, so a worker claims a task by commenting on its issue, as in human collaborations [20, 21].

Party	Command	What it does
Orchestrator	<code>choir orch poll</code>	Blocks until a task or a pull request changes, then exits with a JSON summary of what changed. The loop's clock.
	<code>choir orch create-task</code>	Publishes a task issue from a plan node
	<code>choir orch prs</code>	Open pull requests, with the verification gate's results
	<code>choir orch merge close</code>	Merges a pull request, or closes it unmerged. merge refuses unless the gate passes.
	<code>choir orch comment</code>	Posts a comment on a task issue or a pull request
	<code>choir orch sync-leases</code>	Reads the comments from workers and updates task labels
	<code>choir orch sync-graph</code>	Rederives the roadmap graph's dependency edges from the built project
	<code>choir orch inventory scan</code>	Every axiom and placeholder in the project with locations
	<code>choir orch metrics struggle</code>	Per-task attempt history, recomputed from GitHub
	<code>choir orch salvage</code>	Opens a follow-up seeded from a failed pull request
	<code>choir orch viz</code>	Renders the project's history (Section 4)
Worker	<code>choir worker list</code>	Claimable tasks
	<code>choir worker claim</code>	Takes the lease and, if won, builds the workspace
	<code>choir worker heartbeat release</code>	Refreshes a lease; hands one back
	<code>choir worker submit</code>	Pushes the branch and opens the pull request

Table 2: Choir's most-used commands.

When multiple workers reach for the same task, the earliest comment wins. A claim that falls silent for a day expires and the task returns to the pool. On a won claim the same call builds the workspace, which is the project at the task's pinned commit on a task branch.

Prove and submit. Proving happens on the contributor's own machine under the contributor's own account. Choir ships no proving agent: a contributor can use any agent or workflow, as long as it follows Choir's task format and submission rules. `choir worker heartbeat` keeps a longer lease alive by editing the worker's own comment. When the proof is done, `choir worker submit` pushes the branch and opens the pull request. `choir worker release` hands a task back cleanly, and a scheduled sweep returns long-quiet claims to the pool.

Decompose. Sometimes a published task is too hard. A worker who finds that the mathematics does not fit one pull request may decompose it further: state the intermediate lemmas alongside the target with placeholder bodies, prove the target from them, and declare what was leaned on in a `choir-reduction` block in the pull request body, naming a parent and a list of children.

The proof tree acquires depth from this mechanism. The orchestrator publishes a skeleton and needs to know the structure of the tree without working out every route through it, and what comes back is a decomposition proposed by whoever actually attempted the mathematics. Once it is accepted, the orchestrator updates the roadmap and the dependency graph, then publishes new tasks against each child. Those tasks may go through the same decomposition loop, and the proof tree can keep growing.

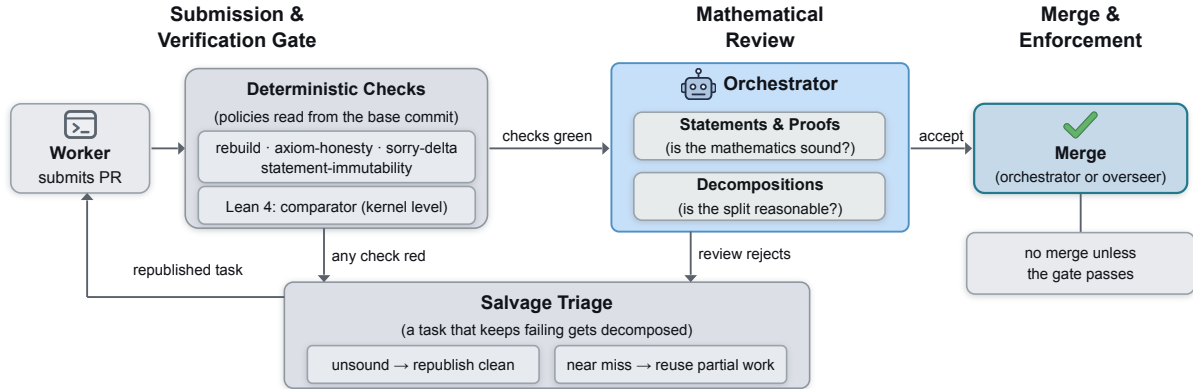


Figure 2: The path of a pull request. The gate’s checks run first, and only a pull request that passes them reaches the orchestrator’s review. When either stage rejects it, the orchestrator can republish the task, from scratch if the work is unsound or from the partial work if it is a near miss.

Review and merge. Each pull request submitted by a worker is first checked by the gate mechanically (Section 3), then reviewed by the orchestrator, which checks whether the mathematics is sound, whether the decomposition is reasonable, and so on (Figure 2). A pull request can only be merged if it passes both the gate and review. Otherwise, the orchestrator can decide to republish the task with or without the partial result from a rejected pull request.

Golf. `golf` is another task type an orchestrator can publish. It pins a declaration that is already proved and asks for a shorter proof of it, with the statement left exactly as it stands. Such a task can come back in one of two ways. The worker may return the shorter proof alone, which is the case the gate verifies best. Or the worker may propose to factor the proof, stating auxiliary lemmas in the same file and naming them in the pull request body.

3 The verification gate

A contributor may be an unknown user running an unknown model. To control the quality of what they submit, a mechanical gate gives every pull request a set of basic checks before it reaches the orchestrator for review. It runs on GitHub Actions, triggered automatically when the pull request is opened or updated, and every check is recomputed repository-side from source under policies resolved at the merge base. The gate is enforced in the `choir orch merge` command, which reads the pull request’s own check rollup and refuses the merge unless every required check is present and every blocking check is green. Below is what the gate checks.

rebuild. Checks that the submission compiles from source in a fresh checkout on Github.

axiom-honesty. Refuses a submission that introduces a new `axiom`.

sorry-delta. Blocks a submission that still leaves a placeholder (`sorry` on Lean 4, `sorry` or `oops` on Isabelle, `Admitted`, `admit`, or `Abort` on Rocq), unless the worker declares the submission

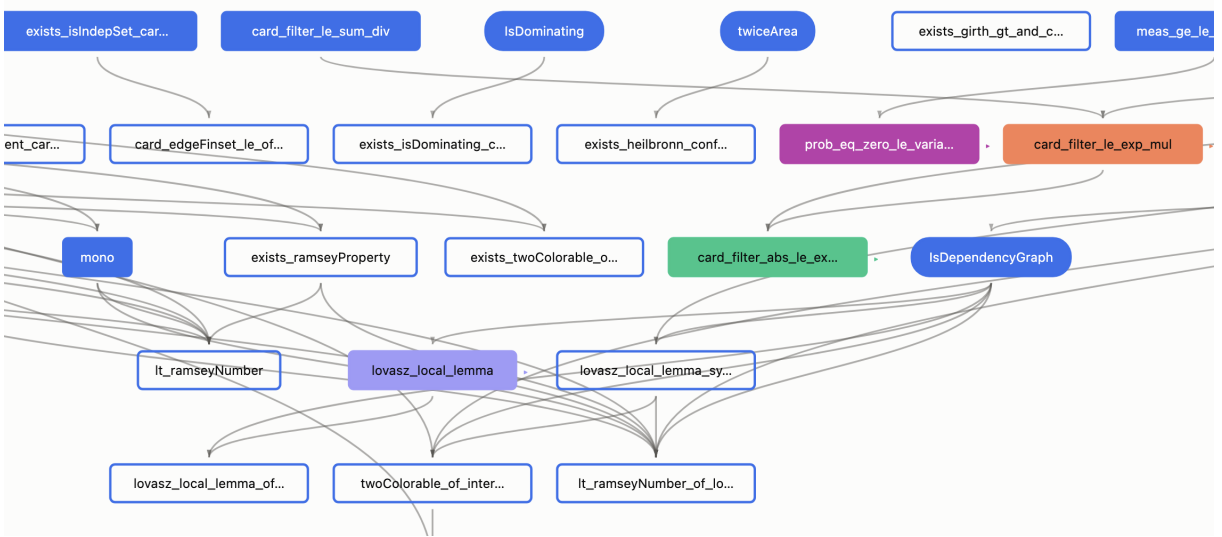


Figure 3: Part of the Zhao formalization’s proof tree in the history visualizer, partway through the project. Blue fill marks proved nodes, a blue outline published tasks, and each other color one worker’s claim; rounded boxes are definitions.

a decomposition (Section 2), in which case the check passes and the placeholders are left to the orchestrator’s review.

statement-immutability. Checks that the submission has not changed the statement the orchestrator published. A worker supplies a proof and adjusts nothing about the mathematics that is being claimed. On Lean 4 this can be settled at the kernel level by the Lean FRO’s **comparator** [22], which follows the statement through everything it depends on, down to each definition’s body.

There is no equivalent tool for Isabelle or Rocq yet, so the obligation is enforced against source text alone. That is weaker than it sounds: a worker can leave a published statement byte-identical while changing what it denotes, for instance by smuggling in assumptions. With a contradictory hypothesis in scope, a worker can even prove statements that are false, by the principle of explosion. Currently, catching this relies on review by the orchestrator and the human overseer.

4 Visualization of the history

An advantage of building on GitHub is that a project’s git log, issues, and pull requests already record how its proof tree was built. **choir orch viz** derives a timeline from those three sources and writes a single HTML page that replays it (Figure 3).

The page draws the proof tree and steps through the project’s dated events, with a slider or a play button: nodes stated or filled, tasks published, claimed, or released, pull requests merged or closed, and decompositions accepted. Each node is colored by whether it is unpublished, published, claimed, partly proved, or proved, and each claim carries its worker’s own hue, so the page shows where each proof and decomposition came from. In Figure 3, four tasks are claimed at once, each by a different worker, while many of the declarations above them are already proved.

Human overseers can use the page to follow a project’s progress.

5 Related work

Most recent autoformalization systems, including Meta FAIR’s RepoProver and AutoformBot [8, 9], are multi-agent: a planner writes a blueprint of statements, workers prove them in parallel, and reviewers check the results before they are merged. LeanMarathon, FormalFlow, and Archon use similar pattern for long research-level proofs [23–25], and other agentic systems target quantum computation, tensor networks, and research papers in general [26–28]. Choir uses the same orchestrator–worker structure, and the difference is who runs the workers. In all of these systems one team runs every agent on its own compute and budget, while Choir’s workers are other people’s agents running on their own machines and accounts.

Pooling contributors’ own resources goes back to volunteer computing. In BOINC, participants donate computing time on their own machines, and because the project has no control over them, it checks the returned results itself [29]. The systems closest to Choir apply the same idea to formalization, with contributors who each run their own agents. TauCeti [17] is a Lean library downstream of Mathlib. Humans write its roadmap and review rubrics, contributors run workers on their own subscriptions, and AI reviewers judge each pull request against the rubrics. Lean Pool [30] is an archive of formalized mathematics that AI agents maintain and outside contributors extend through pull requests, checked by CI and LLM review. Prove2Me [18] is a hosted platform where each theorem is posted once as an immutable statement, and contributors’ agents submit proofs against it. Anthropic’s formal proof of Fermat’s Last Theorem was produced with Prove2Me [16]. In Agent Hunt [31], LLM agents post and claim lemmas through a bounty market. Like TauCeti, Choir distributes work from a roadmap through pull requests on GitHub. Like Prove2Me, it fixes a statement before anyone proves it: the gate checks that a pull request leaves the published statement unchanged. It differs from these in being infrastructure rather than a single library or platform, which any project can set up on its own repository with its own orchestrator. Physlib AI Tools [32] lets a Physlib contributor run Claude Code on their own plan against generic tasks such as golfing a proof. Choir’s golf task is inspired by it, but lets the worker shorten the proof by factoring it into auxiliary lemmas instead, if it is a better choice.

6 Limitations and next steps

Our architecture follows the classic multi-agent design of one orchestrator and several workers. On a project the size of Mathlib, the orchestrator might eventually lose track of the mathematics. To relieve this, the orchestrator states only a shallow skeleton and leaves further decomposition to the workers, and the roadmap is split into groups of related results, so the orchestrator reads only the part of the project a decision needs. We have not tested this beyond textbook scale; if it does not hold, a hierarchy of orchestrators may help.

As Section 3 notes, statement-immutability on Isabelle and Rocq compares source text, so a submission that changes what a statement means without changing its text is caught only in review. We plan to close this gap mechanically in a future version.

So far, Choir has been tested mainly on textbook-level autoformalization in Lean 4, with a few workers running commercial models such as Claude Opus 5 and GPT-5.6. We welcome collaborators for larger-scale runs, including runs on open-source models and harnesses. We expect that large-scale formalization will soon be within reach of academic groups and even

hobbyists, working together on a limited budget with subscriptions and open-source models.

Acknowledgments

The project is sponsored by the Defense Advanced Research Projects Agency under cooperative agreement HR0011262E027. The content of the information does not necessarily reflect the position or the policy of the Government, and no official endorsement should be inferred.

AI disclosure

We used Claude Opus 4.8 and Claude Opus 5 to write the code of Choir, under the authors’ supervision and review throughout development. The architecture was designed and tested by the authors. The authors wrote the paper, drawing on reports that Claude produced during development, and used Claude to polish the wording and fix grammar and typos. Claude also helped with the literature search and with proposing the title. The initial drafts of the figures were generated by Nano Banana 2 and Claude Opus 5 from our instructions, then edited and refined by the authors. The logo was generated with Nano Banana 2 and Claude Opus 5. The authors have reviewed all AI-assisted work, and take full responsibility for the final content of this work, including any text, claims, or artifacts produced with the aid of AI.

References

- [1] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *Lecture Notes in Computer Science*, pages 625–635. Springer, 2021.
- [2] Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL: A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer, 2002.
- [3] The Rocq Development Team. The Rocq prover, 2025. <https://rocq-prover.org>. Accessed 2026-07-02.
- [4] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, pages 367–381. ACM, 2020.
- [5] Physlib contributors. Physlib: Digitalising physics in Lean 4, 2026. GitHub repository, <https://github.com/leanprover-community/physlib>. Formerly PhysLean and HepLean. Accessed 2026-09-23.
- [6] Joseph Tooby-Smith. HepLean: Digitalising high energy physics, 2024. arXiv:2405.08863.
- [7] Clark Barrett, Swarat Chaudhuri, Fabrizio Montesi, Jim Grundy, Pushmeet Kohli, Leonardo de Moura, Alexandre Rademaker, and Sorrachai Yingchareonthawornchai. CSLib: The Lean computer science library, 2026. arXiv:2602.04846. Code: <https://github.com/leanprover/cslib>.
- [8] Fabian Gloeckle, Ahmad Rammal, Charles Arnal, Rémi Munos, Vivien Cabannes, Gabriel Synnaeve, and Amaury Hayat. Automatic textbook formalization, 2026. arXiv:2604.03071. Code: <https://github.com/facbookresearch/repoprover>.
- [9] Ahmad Rammal, Niket Patel, Fabian Gloeckle, Amaury Hayat, Julia Kempe, Rémi Munos, Charles Arnal, and Vivien Cabannes. Formalizing mathematics at scale, 2026. arXiv:2605.29955.
- [10] Zichen Wang, Wanli Ma, Zhenyu Ming, Gong Zhang, Kun Yuan, and Zaiwen Wen. M2F: Automated formalization of mathematical literature at scale, 2026. arXiv:2602.17016.
- [11] Theodore Meek, Siyuan Ge, Di Qiu Xiang, Simon Chess, and Vasily Ilin. Formalizing numerical analysis: An agent pipeline and quality audit beyond kernel acceptance, 2026. arXiv:2606.14000.
- [12] Dustin Bryant, Jonathan Julián Huerta y Munive, Cezary Kaliszyk, and Josef Urban. Munkres’ general topology autoformalized in Isabelle/HOL, 2026. arXiv:2604.07455.
- [13] Josef Urban. 130k lines of formal topology in two weeks: Simple and cheap autoformalization for everyone?, 2026. arXiv:2601.03298.
- [14] Sidharth Hariharan, Christopher Birkbeck, Seewoo Lee, Ho Kiu Gareth Ma, Bhavik Mehta, Auguste Poiroux, and Maryna Viazovska. Progress in formalizing sphere packing in dimension 8, 2026. arXiv:2604.23468.
- [15] Math Inc. Completing the formal proof of higher-dimensional sphere packing, 2026. Blog post, February 2026, <https://www.math.inc/sphere-packing>.

- [16] Anthropic. Formalizing Fermat’s last theorem, 2026. September 4, 2026, <https://www.anthropic.com/research/formalizing-fermats-last-theorem>. Proof: <https://github.com/anthropics/fermats-last-theorem>.
- [17] TauCeti Project. TauCeti: an AIs-welcome Lean library downstream of Mathlib, 2026. GitHub repository, <https://github.com/TauCetiProject/TauCeti>. Apache-2.0. Accessed 2026-07-02.
- [18] Shuze Chen, Kunal Marwaha, Xiaoyang Lu, Henry Yuen, and Tianyi Peng. Prove2Me: An open collaborative platform for scaling math formalization, 2026. arXiv:2608.28433. Platform: <https://prove2.me>.
- [19] Yufei Zhao. Probabilistic methods in combinatorics, 2024. Lecture notes for MIT 18.226 (Fall 2022), last updated June 18, 2024. MIT OpenCourseWare, <https://ocw.mit.edu>.
- [20] Kevin Buzzard and Richard Taylor. FLT: An ongoing Lean formalization of Fermat’s Last Theorem, 2025. GitHub repository, <https://github.com/ImperialCollegeLondon/FLT>. Accessed 2026-09-24.
- [21] Matthew Bolan, Joachim Breitner, Jose Brox, Nicholas Carlini, Mario Carneiro, et al. The Equational Theories Project: Advancing collaborative mathematical research at scale, 2025. arXiv:2512.07087.
- [22] Lean FRO. comparator: A trustworthy judge for Lean proofs, 2026. GitHub repository, <https://github.com/leanprover/comparator>. Apache-2.0. Accessed 2026-07-16.
- [23] Yuanhe Zhang, Yuekai Sun, Taiji Suzuki, Jason D. Lee, and Fanghui Liu. LeanMarathon: Toward reliable AI co-mathematicians through long-horizon Lean autoformalization, 2026. arXiv:2606.05400.
- [24] Sirui Lu, Ruixuan Deng, Yanqiao Zhu, and Zhengfeng Ji. Long-horizon autoformalization of a core theorem underlying $MIP^* = RE$, 2026. arXiv:2609.19814.
- [25] Haocheng Ju, Guoxiong Gao, Jiedong Jiang, Bin Wu, et al. Automated conjecture resolution with formal verification, 2026. arXiv:2604.03789.
- [26] Yuanjie Ren, Jinzheng Li, and Yidi Qi. MerLean: An agentic framework for autoformalization in quantum computation, 2026. arXiv:2602.16554.
- [27] Sirui Lu, Erickson Tjoa, and J. Ignacio Cirac. Multi-agent autoformalization of tensor network theory, 2026. arXiv:2607.07857.
- [28] Arshia Soltani Moakhar, Iman Gholami, Max Springer, Mahdi JafariRaviz, and MohammadTaghi Hajiaghayi. Beyond the library: An agentic framework for autoformalizing research mathematics, 2026. arXiv:2606.31134.
- [29] David P. Anderson. BOINC: A system for public-resource computing and storage. In *Proceedings of the 5th IEEE/ACM International Workshop on Grid Computing (GRID 2004)*. IEEE, 2004. doi:10.1109/GRID.2004.14.
- [30] Vasily Ilin. Lean Pool: An AI-maintained archive of formalized mathematics, 2026. arXiv:2609.25199.
- [31] Chad E. Brown, Cezary Kaliszyk, and Josef Urban. Agent Hunt: Bounty based collaborative autoformalization with LLM agents, 2026. arXiv:2603.06737.
- [32] Joseph Tooby-Smith. Physlib AI tools, 2026. GitHub repository, <https://github.com/jstoobysmith/PhyslibAITools>. Accessed 2026-09-23.